

Museum of Broken Relationships

Relatório Técnico

Unidade Curricular: Desenvolvimento de Aplicações Web

Ano Letivo: 2025/2026

Grupo: Mariana Antunes, Carolina Fernandes e Pedro Figueiredo

1. Introdução

O projeto Museum of Broken Relationships é um jogo web de gestão de recursos. O tema escolhido é a recuperação emocional depois do fim de uma relação. O jogador entra num museu simbólico e vai preparando espaços de cura, tomando decisões e recolhendo resultados.

O objetivo principal é aplicar os conteúdos trabalhados nas aulas de Desenvolvimento de Aplicações Web: HTML, CSS, JavaScript, Flask, SQLite, autenticação, sessões, padrão MVC e comunicação entre frontend e backend.

2. Tema e Mecânica

O jogador gere dois recursos principais:

- Amor-Próprio
- Lágrimas

Cada novo utilizador começa com 4 pontos de Amor-Próprio e 60 Lágrimas. As Lágrimas são gastas para preparar espaços e tomar decisões. O Amor-Próprio muda de acordo com as escolhas feitas nas tarefas.

O jogo possui quatro espaços:

- Baú das Recordações Físicas
- Arquivo Digital
- A Mente e os Pensamentos
- Novos Começos

Cada espaço tem um custo, um tempo de preparação e várias tarefas. Depois de uma tarefa terminar, o jogador precisa de carregar no botão de recolha para receber o resultado.

3. Requisitos Implementados

O projeto implementa os requisitos mínimos obrigatórios do enunciado:

- Sistema de recursos com dois recursos principais
- Recursos iniciais no registo
- Mais de três construções/espacos
- Slots de construção por utilizador
- Tempo de construção
- Tarefas com tempo
- Recolha manual
- Dashboard principal
- Autenticação com registo, login e logout
- Sessões com Flask-Login
- Base de dados SQLite
- Frontend com HTML, CSS e JavaScript
- Comunicação com o backend usando Fetch API
- Histórico de ações recentes
- Leaderboard

4. Arquitetura MVC

A aplicação segue o padrão MVC:

- Model: `models/game_model.py`
- View: `templates/` e `static/`
- Controller: `controllers/main_controller.py`

O ficheiro `app.py` inicia a aplicação Flask, configura o `LoginManager` e regista as rotas.

O ficheiro `settings.py` guarda configurações simples da aplicação, como a porta, o modo debug e a chave secreta.

5. Base de Dados

A base de dados usada é SQLite. O projeto cria automaticamente as tabelas necessárias na primeira execução.

As tabelas principais são:

Tabela | Função

USER | Guarda utilizadores, password, email e recursos

SLOT | Guarda os espaços de cada utilizador

HISTORICO | Guarda ações recentes do utilizador

A tabela USER guarda os dados de autenticação e os recursos atuais. A tabela SLOT guarda o estado de cada espaço: vazio, construindo, ativo, processando, concluída ou finalizado. A tabela HISTORICO permite mostrar no dashboard as ações recentes.

6. Backend

O backend foi desenvolvido em Flask. As rotas principais são:

- /
- /register
- /login
- /logout
- /dashboard
- /construir
- /dar-ordem
- /recolher
- /reiniciar

Também existem rotas usadas pelo JavaScript através de Fetch API:

- /api/construir
- /api/dar_ordem
- /api/recolher
- /api/estado

As rotas normais com POST permitem que o jogo continue a funcionar com formulários. As rotas API permitem ao frontend atualizar dados sem depender apenas de recarregar a página.

7. Frontend

O frontend usa templates Jinja, HTML, CSS e JavaScript.

As páginas principais são:

- index.html
- login.html
- register.html
- dashboard.html
- base.html

O dashboard apresenta os recursos, os espaços de cura, o estado de cada espaço, o leaderboard e o histórico. O ficheiro static/js/jogo.js usa Fetch API para comunicar com o backend e atualizar recursos, notificações e barras de progresso.

O ficheiro static/js/tema.js permite alternar entre tema claro e escuro, guardando a escolha no localStorage.

8. Autenticação

O sistema de autenticação usa Flask-Login. O utilizador pode criar conta, entrar, sair e manter a sessão entre páginas.

As passwords são guardadas com hash através da biblioteca Passlib, como explicado nos laboratórios.

9. Fluxo do Jogo

O fluxo principal é:

1. O utilizador regista-se ou faz login.
2. Entra no dashboard.
3. Prepara um espaço gastando Lágrimas.
4. Aguarda o tempo de preparação.
5. Escolhe uma tarefa/decisão.
6. Aguarda o tempo da tarefa.
7. Recolhe o resultado.
8. Avança até completar os espaços.

Quando todos os espaços são concluídos, o Amor-Próprio chega a 100.

10. Documentação

O projeto inclui:

- README com instruções de execução

- Documentação das rotas em `api_documentacao.md`
- Esquema SQL em `base_dados.sql`
- Relatório técnico em PDF
- Código organizado por pastas

11. Conclusão

O projeto cumpre os requisitos mínimos obrigatórios do enunciado. A aplicação possui backend em Flask, base de dados, autenticação, dashboard, recursos, construções, tarefas com tempo, recolha manual, histórico e comunicação com Fetch API.

O resultado final é uma aplicação web funcional, simples e coerente com os conteúdos trabalhados nos laboratórios.